Distributed Rate Limiting: Redis, Lua & 429 Headers

one limit across thirty servers, and what to do when Redis is down

Watch the 4-min video

> **In one line:** keep per-tenant counters in a shared store like Redis, update them atomically with a Lua script (optionally with local buckets that sync in batches), reject with 429 plus Retry-After and rate limit headers, and decide deliberately whether to fail open or closed when Redis is down.

1 Why local counters are not enough

Your API gateway runs **30 instances** behind a load balancer. Each tenant may make **1,000 requests per minute**. If every instance counts on its own, a tenant whose traffic spreads across all thirty can actually make **30,000**. The limit only works if the instances **share state**.



Figure 1. Left: 30 independent counters allow 30× the limit. Right: one shared counter per tenant in Redis enforces the real limit.

2 A shared counter in Redis, and its races

The classic answer is a shared counter in Redis. For a fixed window, each instance runs `INCR` on a key built from the tenant and the current minute, such as `rl:tenant42:202611041200`. If the result is 1 (the first request this minute), it also runs `EXPIRE` so the key cleans itself up. If the count goes over 1,000, it returns 429. `INCR` itself is atomic, so concurrent increments never get lost.

But there are two races:

- **INCR and EXPIRE are two separate commands.** If an instance crashes (or the connection drops) between them, the key never expires. Its count stays above 1,000, and that tenant stays blocked forever.
- **Read-then-write logic** (read the count, decide in the app, then write) lets two instances both read 999 and both allow.

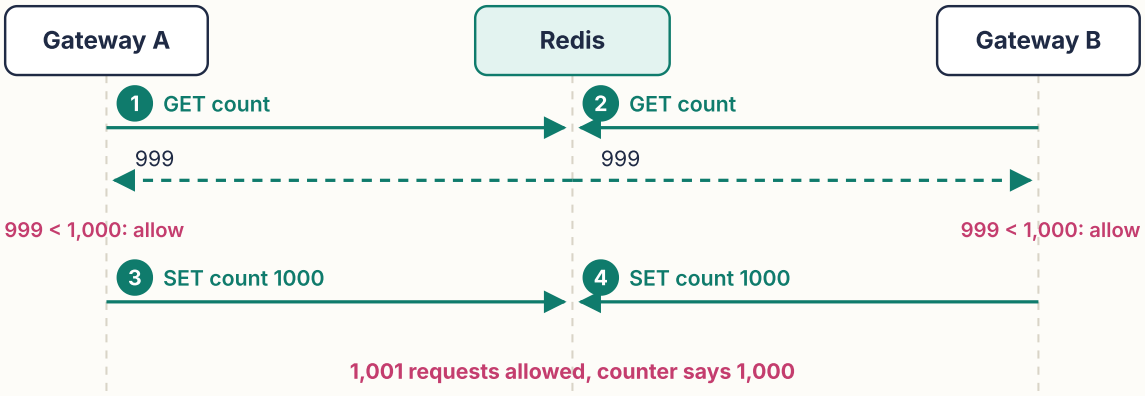


Figure 2. The read-then-write race. Both gateways decide on the same stale value, so the limit is exceeded. Under real load, many requests slip through this gap.

3 The fix: atomicity with a Lua script

Put the whole algorithm in a **Lua script**. Redis runs a script as one indivisible operation: no other command runs on the server while it executes. The script reads the bucket, refills tokens, takes one, sets the TTL and returns allowed or denied, all in **one network round trip**. Load it once with `SCRIPT LOAD` and call it by its hash with `EVALSHA`, so you don't resend the code each time.

```
-- token bucket: KEYS[1] = "rl:{tenant42}"   ARGV = capacity, tokens per ms, now (ms)
local cap, rate, now = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3])
local b = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(b[1]) or cap            -- new key: start with a full bucket
local ts     = tonumber(b[2]) or now
tokens = math.min(cap, tokens + math.max(0, now - ts) * rate)   -- lazy refill
local allowed = 0
if tokens >= 1 then tokens = tokens - 1; allowed = 1 end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', KEYS[1], math.ceil(cap / rate))  -- idle key cleans itself up
return {allowed, math.floor(tokens)}
```

With **Redis Cluster**, keys are sharded by tenant, so the load spreads across nodes. A script may only touch keys in one hash slot; a hash tag like `{tenant42}` keeps all of one tenant's keys together. Pass the time in from the gateway, or read it inside the script with `redis.call('TIME')` so clock skew between gateways can't distort the refill.

4 Cutting the cost: local buckets that sync

A Redis call on every request adds a round trip (often 0.5–1 ms in the same zone) and makes Redis a hot dependency on the critical path. Two common optimizations:

- **Hybrid local + global.** Each instance keeps a local token bucket and syncs with Redis in batches, for example claiming **50 tokens at a time**. Only one request in fifty touches Redis: at 60,000 requests per second, Redis sees about 1,200 calls per second instead of 60,000.
- **Split the limit.** With 30 instances, each allows about 1,000 / 30 ≈ **33 per minute** locally, with no Redis at all. It works well only when the load balancer spreads each tenant's traffic evenly.

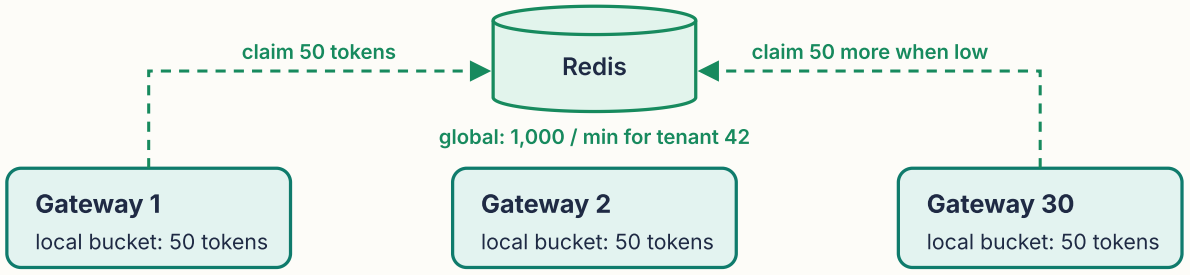


Figure 3. Hybrid limiting: gateways spend tokens locally and claim a new batch from Redis when they run low.

You trade a little accuracy for speed and resilience. Claimed tokens are already deducted in Redis, so the long-run total stays right, but timing blurs: up to 30 × 50 = 1,500 tokens can sit parked on gateways. One gateway may reject a tenant while another holds spare tokens, and parked tokens can be spent later in a burst.

5 Tell clients what's happening

When you reject, return **429 Too Many Requests** with a `Retry-After` header in seconds. On **every** response, include the limit, the remaining count and the reset time, so well-behaved clients slow down before they hit the wall:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 12
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1762257660      <- when the window resets (Unix time)

# newer IETF draft (httpapi-ratelimit-headers) style:
RateLimit-Policy: "default";q=1000;w=60
RateLimit: "default";r=0;t=12
```

The `X-RateLimit-*` headers are a widespread convention (GitHub and many others use them). An IETF draft standardizes them as `RateLimit` and `RateLimit-Policy`.

6 What if Redis is down?

	Fail open	Fail closed
Behaviour	Allow requests; local in-memory limits act as a backstop	Reject requests (429 or 503)
Protects	Availability of your API	A fragile backend from overload
Cost	Limits are approximate for a while	A Redis outage becomes a full API outage
Typical for	Most public APIs	Expensive or abuse-prone paths (login, SMS sending, payments)

Most public APIs **fail open with local fallback limits, and alert loudly**. Either way, put a **short timeout** on the Redis call (a few milliseconds) and a **circuit breaker** around it, so a slow Redis can't slow down every request. A hung Redis is worse than a dead one if every request waits a full second for it.

COMMON MISTAKES

- Rate limiting in each instance's memory and calling it a global limit.
- Running `INCR` and `EXPIRE` as separate calls with no recovery: a crash in between blocks a tenant forever.
- Returning 429 without `Retry-After`, so clients retry immediately and make it worse.

7 Key terms

Shared state: one counter all instances see

INCR: atomic Redis increment

Lua script: runs atomically inside Redis

EVALSHA: call a loaded script by hash

Hash tag: {...} keeps keys in one cluster slot

Hybrid limiter: local buckets + batched sync

Retry-After: seconds to wait before retrying

Fail open: allow when the limiter is down

Fail closed: reject when the limiter is down

8 Interview questions

Q1. How do you rate limit across many servers? BASIC

Keep counters in a shared store like Redis, keyed per tenant, updated atomically with a Lua script. Optionally use local buckets that sync in batches. Reject with 429, Retry-After and rate limit headers.

Q2. Why use a Lua script instead of separate Redis commands? BASIC

Redis runs the script atomically, so read-refill-take-write can't interleave with another gateway, and it costs one round trip. Separate commands leave gaps: two gateways read 999 and both allow, or a crash between INCR and EXPIRE leaves a key that never expires.

Q3. You can't afford a Redis call per request. What do you do? SCENARIO

Hybrid: each gateway claims tokens from Redis in batches (say 50) and spends them locally, cutting Redis calls 50×. Or split the limit: about 33 per gateway per minute, if traffic is evenly balanced. Accept slightly blurred accuracy.

Q4. What happens if Redis is down? SCENARIO

Usually fail open with local per-instance limits as a fallback, alert, and guard the call with a short timeout and a circuit breaker. Fail closed only where protecting the backend matters more, such as login or SMS sending.

Q5. A tenant stays blocked for hours after they stopped sending. Why? SCENARIO

A counter key with no TTL: INCR ran, then the app crashed before EXPIRE, so the count stays over the limit forever. Do INCR and EXPIRE in one Lua script.

Q6. Redis becomes slow (200 ms per call) but not down. What breaks? TRICKY

Every request now waits 200 ms more, so API latency and in-flight requests jump (Little's Law), and pools may run dry. A tight timeout (say 5 ms) plus a circuit breaker turns slow into “unavailable”, so the fail-open path kicks in.

Q7. Is the hybrid (batched local tokens) design exact? TRICKY

Not quite. The long-run total holds, but up to 30 × 50 = 1,500 tokens can be parked on gateways: one may reject the tenant while others hold spares, or spend them in a later burst. Smaller or expiring batches shrink the error, at the cost of more Redis calls.

Next: SD-C-014 · Throttling vs Rate Limiting vs Quotas backendroadmap.com